



GradeForge
offline & explainable

PROJECT PROPOSAL · SEPTEMBER 2026

GradeForge

An offline, explainable AI system for grading written exam answers — built to save teachers time without asking them to trust a black box.

100%

OFFLINE & LOCAL

5

STRICTNESS POLICIES

3

EXPORT FORMATS

∞

LANGUAGES SUPPORTED

EXECUTIVE SUMMARY

Grading that explains itself

Written exams — short answers, essays, open-ended science and humanities questions — remain the most time-consuming part of a teacher's workload, and the hardest to grade consistently across a full class. GradeForge is a desktop application that reads an answer key and a stack of student papers, semantically compares each answer against the official one, and produces a score together with the specific evidence behind it: which concepts were covered, which were missing, whether the answer contradicts the key, and how confident the system actually is.

The product runs entirely offline. There is no student data sent to any server, no subscription, and no dependency on an internet connection once the software and its language model are installed. Every score the system produces is accompanied by the reasoning behind it, and anything the system is not confident about is routed to a human reviewer rather than silently guessed at.

"If the system does not have enough evidence to confidently grade an answer, it must expose that uncertainty instead of pretending to be certain."

— Core design principle behind GradeForge

THE PROBLEM

Manual grading does not scale

Multiple-choice exams are trivial to grade by machine. Written answers are not — and that gap pushes real cost onto teachers and real inconsistency onto students:

Time

Grading 30–150 written answers per question, per class, repeatedly across a term, consumes hours that could go into actual teaching and feedback.

Consistency

The same answer can receive a different grade depending on the time of day, fatigue, or which paper was read immediately before it.

Opacity

A number on a page rarely tells a student why they lost the points, which is exactly the feedback that would help them improve.

Language

Multilingual classrooms and non-English curricula are underserved by the grading tools that do exist, most of which assume English and multiple-choice formats.

Existing “AI grading” products that do address written answers are almost universally cloud-based, which raises real questions for schools about student-data privacy, ongoing cost, and what happens when the connection — or the vendor — goes away. GradeForge is designed as the opposite of that: a tool a school or an individual teacher can own outright.

THE SOLUTION

What GradeForge actually does

GradeForge combines a semantic embedding model, a transparent scoring policy, and a guided desktop workflow so a teacher can go from a raw answer key and a folder of student papers to graded, explained, exportable results — without writing a line of code or trusting an opaque score.

Semantic, not keyword, matching

Answers are compared by meaning using multilingual sentence embeddings, so a correct answer phrased differently from the key still earns credit.

Configurable strictness

Five grading policies, from Very Lenient to Very Strict, each mapped to explicit, visible numbers — coverage thresholds, contradiction penalties — never a hidden rule.

Contradiction detection

Flags answers that state the opposite of what the official answer claims, a failure mode plain similarity scoring misses entirely.

OCR for scanned papers

Handwritten or scanned submissions go through OCR with a visible confidence score, so low-quality scans are flagged rather than silently mis-graded.

Human-in-the-loop review

Every low-confidence result, contradiction, or uncertain identity match lands in a dedicated Review Center — the AI grade is never final without that safety net.

Multilingual by design

Built and tested against both English and Persian answer sheets, with automatic language detection and RTL-aware report generation.

Class analytics

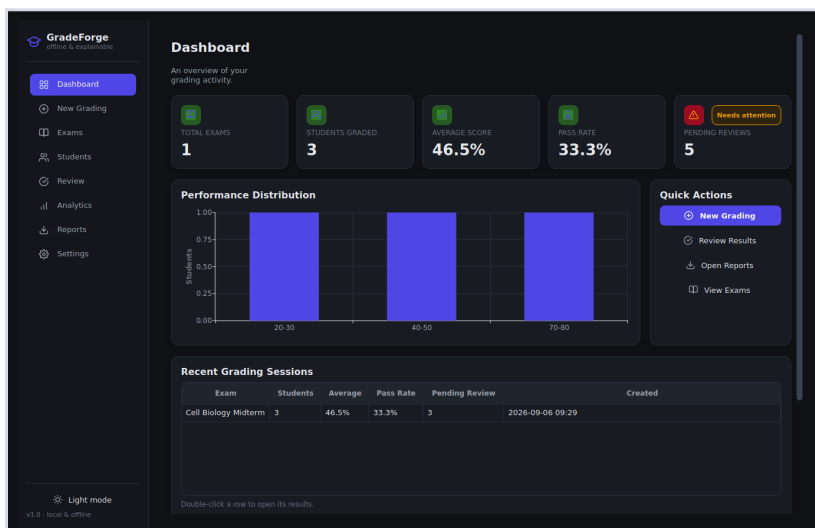
Score distributions, hardest questions, and pass rates, computed from the same results a teacher already reviewed — not a separate, disconnected dashboard.

Offline-first AI

The semantic model can be downloaded once and used forever afterward with zero network access — verified in this build down to the network calls it does not make.

PRODUCT TOUR

A guided workflow, not a script



The dashboard — class performance, pending reviews, and recent sessions at a glance.

The New Grading wizard consists of three steps:

- Exam Setup** (Current step):
 - Exam title:** e.g. Biology Midterm
 - Language:** Auto-detect
 - Description:** Optional note for ...
 - Default max score (if not detected):** 1.00
 - Answer Key:** Drag & drop the answer key here (TXT, DOCX, PDF, PNG, JPG, GIF, BMP). [Browse Files](#)
- Student Papers**
- Grading Settings**

Detected Questions table:

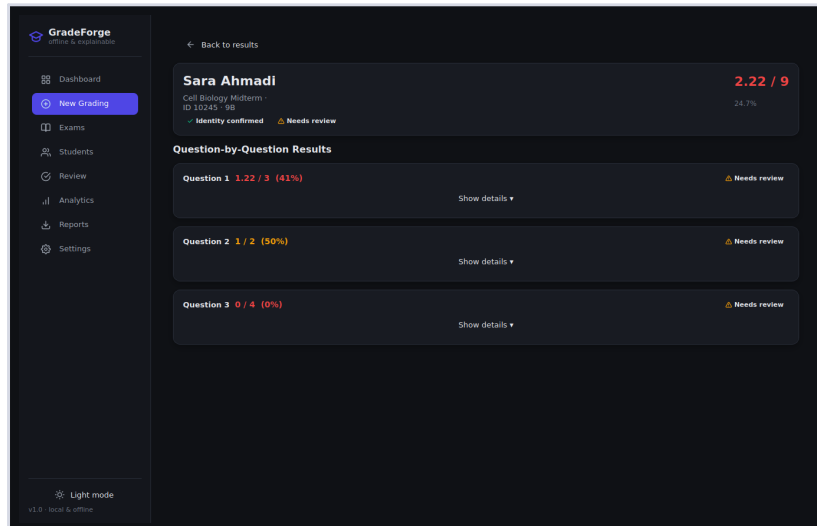
#	Question	Official Answer	Max Score
---	----------	-----------------	-----------

Buttons: Back, Next.

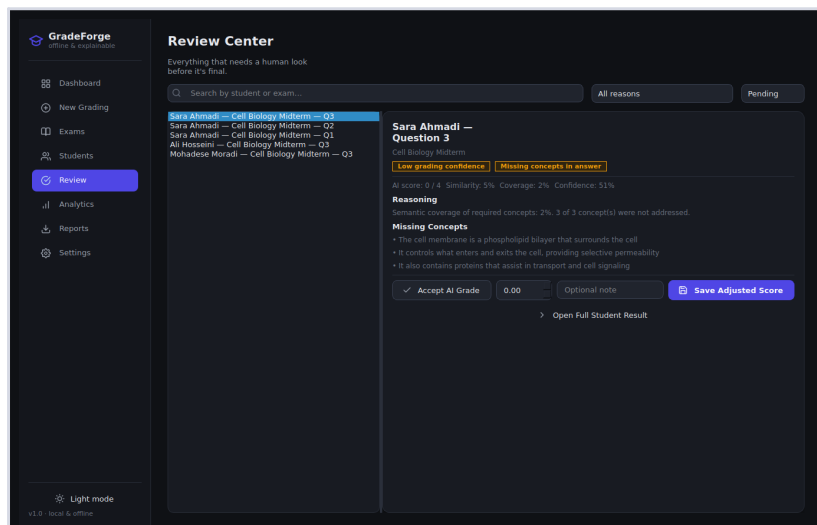
New Grading — a three-step wizard: import the answer key, add student papers, choose a strictness policy.

PRODUCT TOUR

Explainable by construction



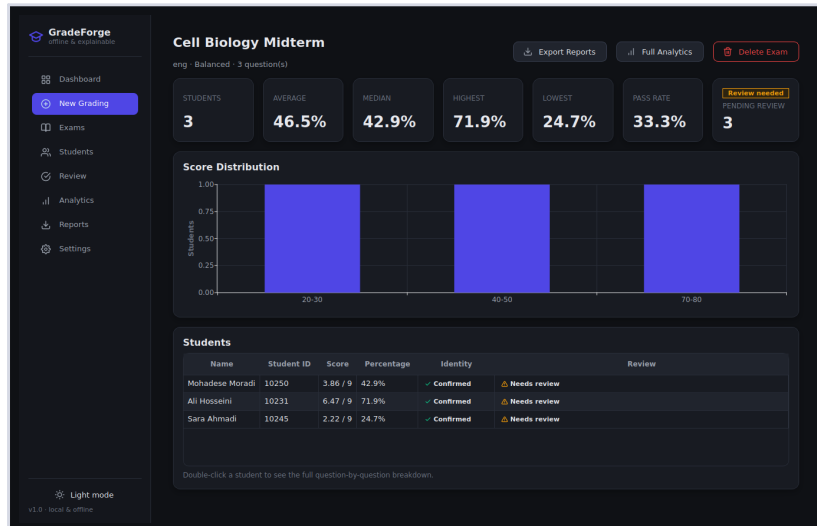
Every question shows its score, similarity, missing concepts, and reasoning — never a bare number.



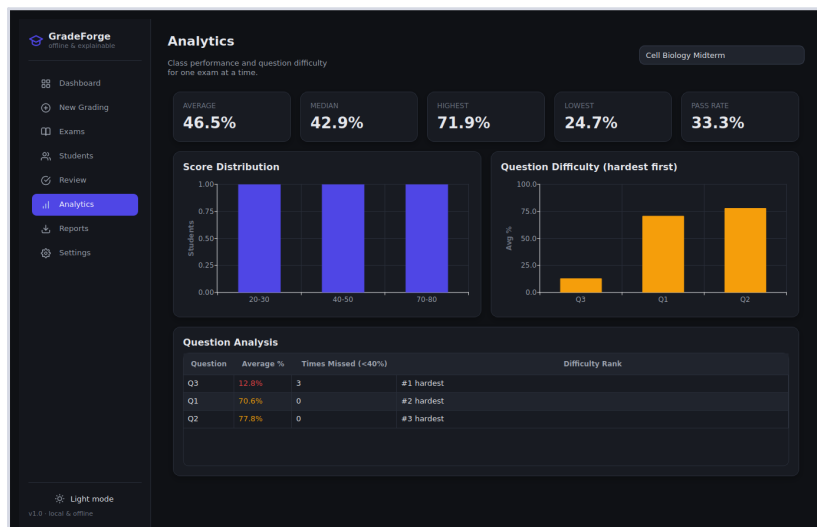
The Review Center: everything the system flagged as uncertain, with the evidence needed to decide fast.

PRODUCT TOUR

From results to insight



Per-exam results overview — score distribution and every student's status in one place.



Class analytics — question difficulty ranked automatically from the same graded results.

ARCHITECTURE

A clean, layered system

GradeForge is built as four strictly separated layers. The interface never contains grading formulas or SQL, and the grading engine has no knowledge that a UI exists — the same engine backs both the desktop application and a scriptable command-line interface used for batch/CI workflows.

Desktop UI	PySide6 — dashboard, guided grading wizard, review center, analytics, reports
Service Layer	Threaded workers for OCR / parsing / grading — the UI never blocks, never touches SQL
Grading Engine	Embedding backend · strictness policy · contradiction detection · confidence scoring
Data & I/O	SQLite (results, review state) · OCR · PDF/DOCX/image parsing · PDF/Excel/JSON export

Data flows strictly downward for input and upward for results: the UI calls the service layer, the service layer calls the grading engine and repository, and nothing skips a layer. This is what makes the CLI and the desktop app produce identical results from identical input — they are calling the exact same code.

TECHNOLOGY

Built on proven, inspectable tools

CATEGORY	TECHNOLOGY	PURPOSE
Application	Python 3.12, PySide6 (Qt 6)	Cross-platform desktop UI with native performance and threading
Semantic AI	sentence-transformers, multilingual-e5-large / bge-m3	Meaning-based answer comparison across 100+ languages
Fallback engine	scikit-learn (TF-IDF)	Automatic, clearly-labeled offline fallback if the transformer model isn't installed
OCR	Tesseract OCR, pdf2image, OpenCV	Text extraction from scanned papers and images, with confidence scoring
Storage	SQLite	Local, file-based, zero-configuration — no database server to run or secure
Reports	ReportLab, openpyxl	Per-student PDF reports, class Excel reports, and raw JSON export
Parsing	python-docx, pypdf	Reading answer keys and submissions from DOCX, PDF, and plain text

Every dependency above is open-source and independently auditable. Nothing in the grading path is a proprietary or hosted API — a school's IT department can read every line involved in producing a student's grade.

METHODOLOGY

A score you can audit, not just trust

Each answer is scored from four transparent signals, combined by a policy whose numbers are shown directly in the product — never hidden behind a single opaque “AI confidence” figure:

Semantic similarity

How close the student's answer is in meaning to the official answer, independent of exact wording.

Concept coverage

What fraction of the key concepts in the official answer were actually addressed.

Contradiction check

Whether the student's answer asserts something that directly conflicts with the official answer.

Confidence

The system's own estimate of how reliable this particular score is — low confidence routes the answer to human review instead of finalizing a guess.

Five strictness policies (Very Lenient through Very Strict) tune how these four signals combine into a final score, and every parameter — coverage needed for full credit, the contradiction penalty, the minimum similarity to count as an attempt — is visible to the teacher before grading begins, with a plain-language explanation of what each policy rewards and penalizes.

STATUS & ROADMAP

What's built, what's next

Delivered

- Core grading engine: OCR, parsing, semantic scoring, contradiction detection, analytics, reports
- SQLite persistence with full grading history and a human-review audit trail (AI score vs. teacher-adjusted score, always kept distinct)
- Full PySide6 desktop application — dashboard, guided grading wizard, review center, analytics, settings
- Command-line interface for scripted / batch grading, sharing the exact same engine as the desktop app
- Verified fully-offline operation path for the semantic model, including a one-time download helper

Next

- Handwriting-specialized OCR (a dedicated model swapped in behind the existing OCR interface)
- Teacher-authored rubrics feeding concept extraction directly, instead of automatic sentence-splitting
- Packaged, double-click installers for Windows and macOS
- Automated test coverage for the parsing and OCR layers, and continuous integration

CLOSING

An assistant, not an authority

GradeForge does not aim to replace a teacher's judgment — it aims to make that judgment faster to apply and easier to explain. Every score comes with its reasoning attached, every uncertainty is surfaced rather than hidden, and every AI decision remains visibly separate from and overridable by the teacher who has the final word.

That combination — genuinely useful automation, full transparency, and complete data ownership — is the product being proposed here, and the majority of it is already built, tested, and running.

"I can give this to a teacher, upload an answer key and 30 student papers, configure how strict the grading should be, click Grade, and immediately understand both the results and why the system produced them."

— The product target this proposal is built against